YOLO 与 ByteTrack

如何实现篮球运动跟踪?

YOLO 找位置 → ByteTrack 认关联 → 绘制轨迹

Python · OpenCV · Ultralytics 内置 ByteTrack

以当前篮球专用权重与视频文件处理为例

01 为什么检测之后，还需要跟踪?

输入视频

连续的视频帧

→

YOLO 检测

每帧球框、分数

→

ByteTrack

关联目标、保持 ID

→

轨迹绘制

连接历史中心点

检测回答“球在哪里”；跟踪回答“前后帧是不是同一个球”。

02 YOLO：从一帧画面中找到篮球

提取特征

边缘、纹理、球的形状

→

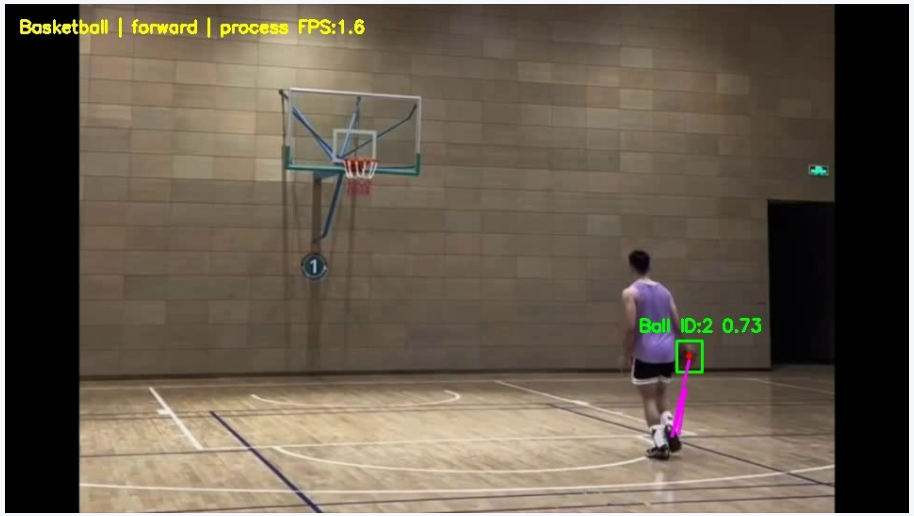
多尺度融合

兼顾远处小球与近处大球

→

输出检测框

位置 + ball 类别分数



当前： YOLOv8n 篮球专用权重

优点：单次前向计算，模型较小

局限：小球、模糊、遮挡仍可能漏检

03 ByteTrack：把前后帧的球“接起来”

① 预测位置

卡尔曼滤波

用位置和速度估计

下一帧球可能在哪里

② 两轮关联

先匹配高分检测

再用低分检测补充

依据框的重叠与线性分配

③ 更新轨迹

匹配成功：延续 ID

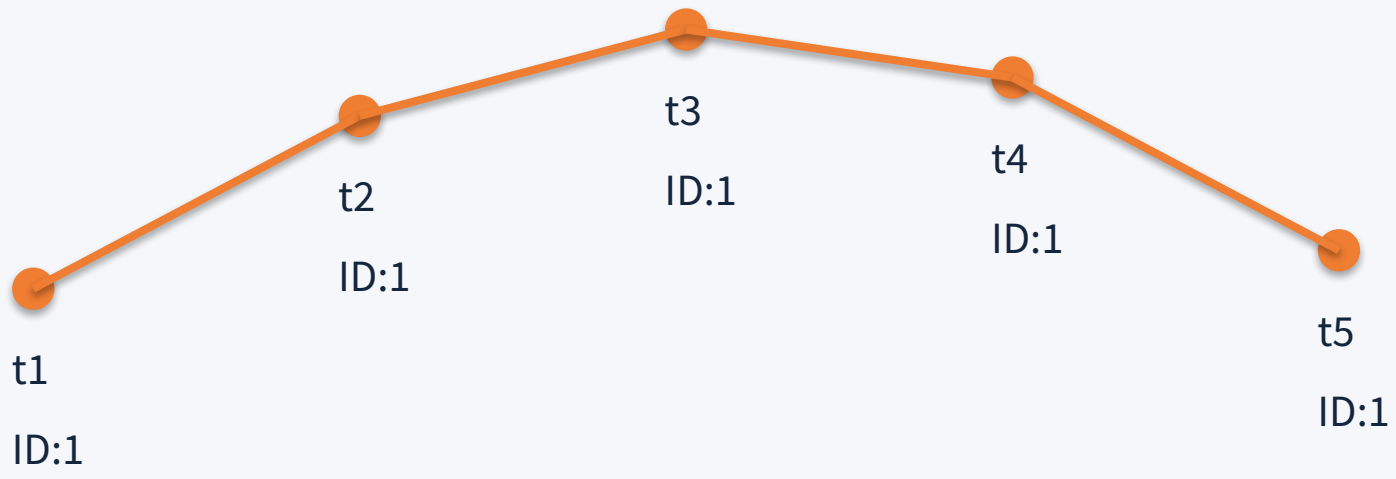
短时丢失：暂时保留

超时丢失：删除轨迹

优点：利用低分检测，缓解遮挡引起的断轨；开销小，无额外 ReID 网络。

04 从目标 ID 到运动轨迹

相同 ID 的中心点按时间连接



清晰显示

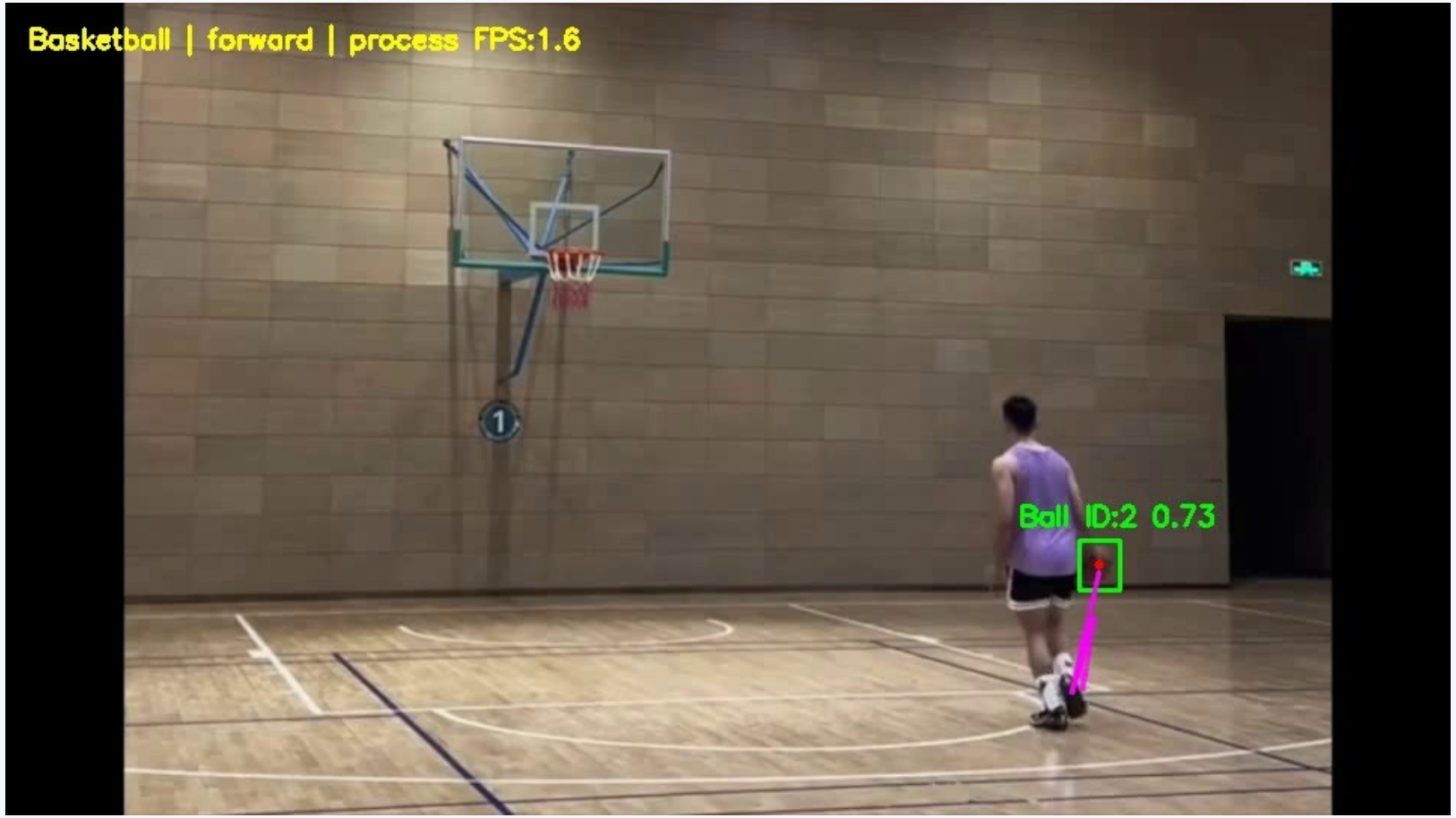
短轨迹 + 淡出

运镜时补偿旧轨迹

遮挡预测用虚线区分

真实检测与预测分开记录：线条连续，并不等于实际跟踪准确。

05 视频演示：检测框、 ID 与轨迹



点击视频画面播放

绿框：真实检测

紫线：历史轨迹

橙色虚线：短时预测

观察：是否跟对、是否断轨

06 已实现什么，还需要改进什么？

当前成果

YOLO 提供位置， ByteTrack 保持 ID

支持结果视频和轨迹记录导出

加入人头排除与相机运动补偿

已具备短时遮挡预测与演示能力

局限与下一步

仍有漏检、人头误检与跨视频失效

反弹、接球后不能保证预测正确

下一步：更多独立标注篮球视频

重点评价跟踪准确性，而非只看连线